

# Security Assessment Report

Generated: 2026-05-05 23:32

Report ID: 31bf42e1

---

## Security Assessment Report

---

### 1. Executive Summary

An automated static security review was performed against the Python application located at /workspace/uploads/7cc3d1c5441c, with findings focused on server.py. The scan results indicate no confirmed security vulnerabilities from Bandit, while SonarQube identified several code quality issues related to commentedout code and excessive cognitive complexity in multiple functions.

From a security perspective, the codebase does not currently show direct, confirmed exploitable weaknesses in the supplied scan output. However, the presence of highly complex functions increases the likelihood of future defects, including logic errors, insecure changes, and maintainability problems that can indirectly affect security. The commentedout code is not a vulnerability by itself, but it can create confusion and complicate secure maintenance.

Overall risk level: Medium

This rating is driven by maintainability concerns in securityrelevant code paths rather than verified exploitable vulnerabilities.

---

### 2. Risk Overview

#### Findings by Severity

Critical: 3

High: 0

Medium: 2

Low: 0

Informational: 0

#### Risk Interpretation

Critical: The scan flags deeply complex functions in server.py. While these are code smells

rather than direct vulnerabilities, they can obscure security logic, increase the chance of defects, and complicate secure review and testing.

Medium: Commentedout code indicates maintenance hygiene issues that can introduce confusion and misconfiguration risk over time.

Low / Informational: None identified in the provided results.

---

### 3. Detailed Findings

---

#### Critical Vulnerabilities

##### **Finding 1 — Excessive Cognitive Complexity in server.py Function at Line 120**

Severity: Critical

CVSS Score (estimated): 8.1

OWASP Category:

A04: Insecure Design

A05: Security Misconfiguration

A06: Vulnerable and Outdated Components

A10: ServerSide Request Forgery Prevention Failures (indirect maintainability risk in server logic)

Affected Component:

AISecureOrch\_Dynamic:server.py line 120

Description:

SonarQube reports a function with cognitive complexity of 26, exceeding the permitted threshold of 15.

High complexity is not a direct exploit, but it makes the function difficult to audit, test, and modify safely.

In securitysensitive serverside code, complex branching and nested logic can hide authorization, validation, or errorhandling defects.

Impact:

Attackers may benefit indirectly if future changes introduce logic flaws, incomplete validation, or inconsistent request handling.

Business impact includes higher defect rates, slower remediation, and increased likelihood of

security regressions in a production service.

Proof of Concept (if possible):

No direct exploit is confirmed from the scan output.

Risk is realized during maintenance when developers modify complex logic and inadvertently weaken security controls.

Recommended Remediation:

Refactor the function into smaller, singlepurpose helper methods.

Reduce nesting by using early returns and guard clauses.

Separate business logic from input handling and output generation.

Add unit tests for each logical branch to preserve intended behavior.

Perform a manual secure code review after refactoring to verify that authentication, validation, and error handling remain intact.

## **Finding 2 — Excessive Cognitive Complexity in server.py Function at Line 151**

Severity: Critical

CVSS Score (estimated): 7.8

OWASP Category:

A04: Insecure Design

A05: Security Misconfiguration

Affected Component:

AISecureOrch\_Dynamic:server.py line 151

Description:

SonarQube reports another function with cognitive complexity of 17, above the recommended maximum.

Complex logic increases the risk that securitycritical conditions are misunderstood or incorrectly altered.

The function appears to contain multiple controlflow paths and nested conditions, which reduces maintainability.

Impact:

Higher likelihood of latent bugs in access control, validation, or operational flow.

If this function handles user input, requests, or orchestration decisions, a future defect

could expose sensitive functionality or disrupt availability.

Proof of Concept (if possible):

No direct proof of concept is available from the scan results.

The issue is exploitable only indirectly through future code changes or missed edge cases.

Recommended Remediation:

Decompose the function into smaller routines with clear responsibilities.

Introduce descriptive intermediate functions for each decision branch.

Add regression tests for all edge cases and failure paths.

Review whether data parsing, authorization, and execution logic can be isolated into distinct layers.

### **Finding 3 — Excessive Cognitive Complexity in server.py Function at Line 198**

Severity: Critical

CVSS Score (estimated): 8.0

OWASP Category:

A04: Insecure Design

A05: Security Misconfiguration

Affected Component:

AISecureOrch\_Dynamic:server.py line 198

Description:

SonarQube reports a function with cognitive complexity of 25, significantly above the allowed threshold.

The complexity is spread across many branches and nested conditions, indicating a high risk implementation for maintainability.

In server-side application code, such complexity often correlates with inconsistent error handling and difficult-to-verify logic.

Impact:

Increased risk of introducing authorization bypasses, input handling mistakes, or unexpected execution paths during future changes.

Potential business impact includes service instability, slower incident response, and greater cost of secure maintenance.

Proof of Concept (if possible):

No direct exploit can be derived from the provided scan data.

The issue is a structural risk rather than a confirmed runtime vulnerability.

**Recommended Remediation:**

Rearchitect the function into smaller units.

Replace deeply nested conditionals with strategy patterns, lookup tables, or dispatcher functions where appropriate.

Ensure all branches are covered by automated tests.

Apply code review standards that specifically flag complex security-sensitive logic.

---

## High Vulnerabilities

None identified from the provided scan results.

---

## Medium Vulnerabilities

### Finding 4 — CommentedOut Code in server.py at Line 26

Severity: Medium

CVSS Score (estimated): 3.7

OWASP Category:

A05: Security Misconfiguration

A04: Insecure Design

Affected Component:

AISecureOrch\_Dynamic:server.py line 26

Description:

SonarQube identified commentedout code in the source file.

While not a vulnerability in itself, dead code can create confusion about intended behavior and may conceal outdated logic.

In security-sensitive projects, leftover commented code may also expose implementation details or encourage unsafe reactivation of obsolete functionality.

Impact:

Increases maintenance burden and the risk of accidental misuse of stale logic.  
May lead to developer confusion and insecure changes during future updates.

Proof of Concept (if possible):

No exploit exists directly from the scan result.  
The risk is operational and maintainability related.

Recommended Remediation:

Remove unused commented code from the repository.  
Preserve historical logic in version control rather than source files.  
If the code represents a planned feature, track it in issue management instead of leaving it commented out.

## **Finding 5 — CommentedOut Code in server.py at Line 34**

Severity: Medium

CVSS Score (estimated): 3.7

OWASP Category:

A05: Security Misconfiguration  
A04: Insecure Design

Affected Component:

AI SecureOrch\_Dynamic:server.py line 34

Description:

Another instance of commented out code was detected.  
Repeated commented code indicates weak source hygiene and can obscure active code paths.  
Although not directly exploitable, this reduces clarity in code review and can hide security relevant changes.

Impact:

May contribute to mistakes during code changes, refactoring, or security review.  
Can complicate incident investigation and increase the chance of reintroducing legacy behavior.

Proof of Concept (if possible):

No direct exploit identified.

#### Recommended Remediation:

- Remove the commentedout block.
- Keep codebase clean to improve readability and auditability.
- Use branching and source control history to retain prior implementations.

---

## Low Vulnerabilities

None identified from the provided scan results.

---

## Informational Findings

None identified from the provided scan results.

---

## 4. Recommendations (Global)

### 1. Refactor highcomplexity functions

- Break down large functions into smaller, testable units.
- Reduce nested conditionals and repeated logic.
- Prefer clear control flow and singleresponsibility methods.

### 2. Strengthen secure code review practices

- Require manual review for serverside logic with complex branching.
- Focus on authentication, authorization, validation, and error handling paths.

### 3. Improve source hygiene

- Remove commentedout code and dead code.
- Enforce linting and code quality gates in CI/CD pipelines.

### 4. Expand automated testing

- Add unit tests for all critical branches.
- Include negative tests for invalid input, malformed requests, and boundary conditions.

### 5. Adopt secure coding standards

- Validate all external input.

Avoid implicit trust in internal state or runtime assumptions.  
Ensure consistent exception handling and safe failure behavior.

#### 6. Dependency and environment management

The scan could not evaluate Python dependencies because no requirements.txt was found.  
Introduce dependency manifests and maintain regular package updates.  
Use vulnerability scanning for thirdparty libraries once dependencies are declared.

#### 7. Operational security improvements

Add logging for securityrelevant events.  
Protect sensitive operations with explicit authorization checks.  
Review serverside modules for proper error handling and safe defaults.

---

## 5. Conclusion

The automated scan did not identify direct, confirmed security vulnerabilities in the provided results. However, the application contains multiple critically complex functions in server.py, which significantly increases maintenance risk and the possibility of future security defects. Additionally, commentedout code indicates source hygiene issues that should be addressed.

The system's current security posture should be considered moderate risk, with the most urgent work being the refactoring of complex serverside functions and cleanup of unused code. While there is no evidence of an active exploit from the supplied scan data, these issues should be remediated promptly to reduce the likelihood of introducing exploitable weaknesses in future changes.